

Assignment 3 - Software Design Patterns

Topic A: Drawing with the Bridge Pattern

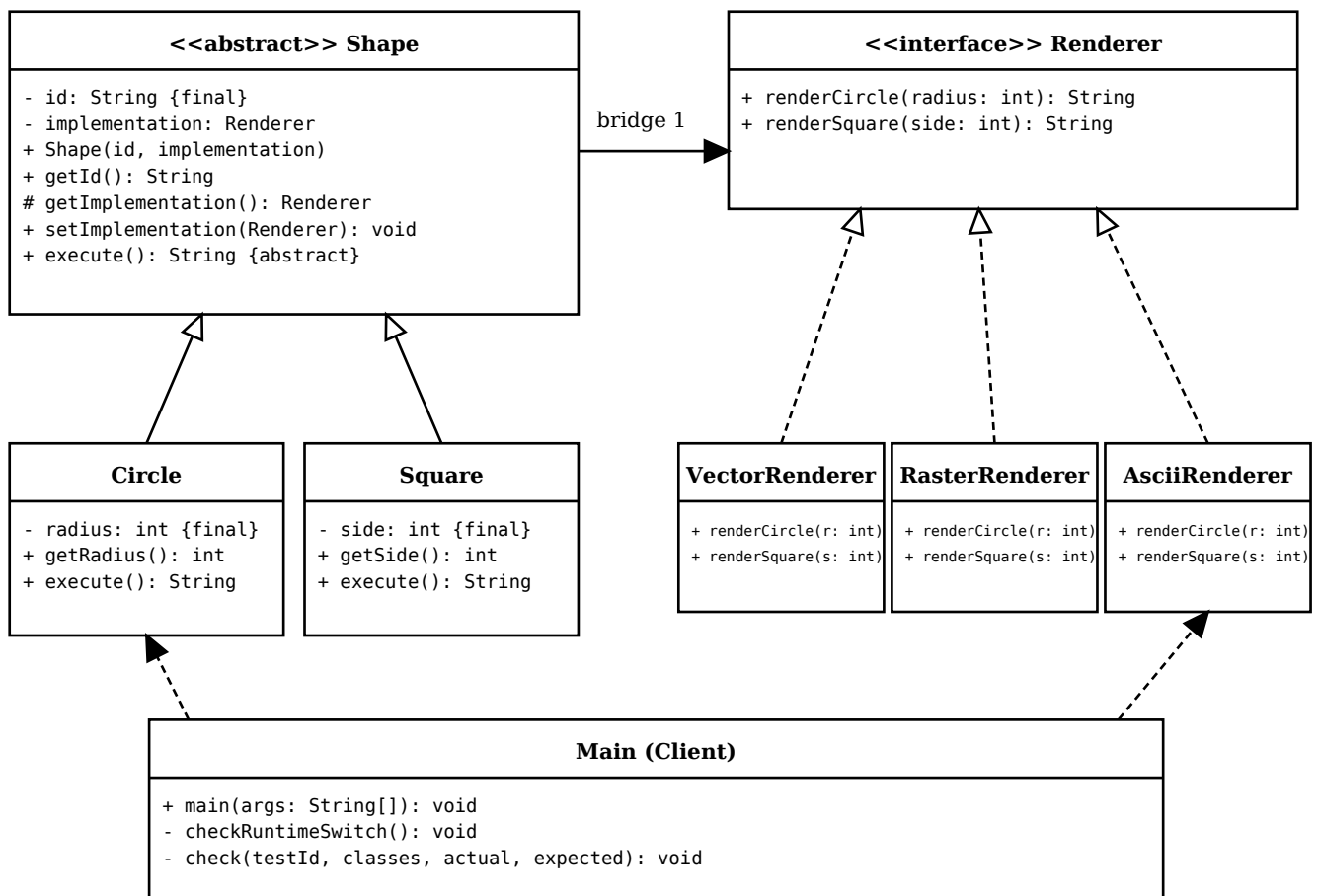
Dauletkyzy Bayan | SE-2523 | 4 October 2026

1. Two independent dimensions

The first dimension is the shape: Circle or Square. Each shape stores its own domain data. The second dimension is the rendering method: VectorRenderer, RasterRenderer or AsciiRenderer. Any shape can use any renderer through the Renderer interface. This produces six combinations without a separate subclass for every combination [1, 2].

Shape stores a private Renderer implementation reference. Its constructor receives this reference, and setImplementation() can replace it. Circle.execute() and Square.execute() delegate rendering through the interface. Shape classes contain no concrete renderer construction, casts or renderer-type branches.

2. UML class diagram



Solid hollow triangles: inheritance. Dashed hollow triangles: interface realization. Dashed arrows from Main: dependencies. The solid arrow from Shape to Renderer is the interface-typed bridge. Renderer implementations return String; constructor parameters follow the source files.

3. Five code excerpts and their benefits

1 - Separate responsibilities (Circle.execute()). Circle chooses its domain operation, while the renderer produces the description. The shape does not decide whether rendering is vector, raster or ASCII.

```
public String execute() {  
    return getImplementation().renderCircle(radius);  
}
```

2 - Meaningful names (Renderer interface). renderCircle, renderSquare, radius and side describe what each operation needs. The interface makes the contract explicit for every implementation.

```
public interface Renderer {  
    String renderCircle(int radius);  
    String renderSquare(int side);  
}
```

3 - Focused methods (Shape.setImplementation()). The setter only replaces the rendering strategy. Objects.requireNonNull rejects a missing renderer immediately, rather than allowing a later rendering failure.

```
public void setImplementation(Renderer implementation) {  
    this.implementation = Objects.requireNonNull(implementation);  
}
```

4 - Avoid repeated testing workflow (Main.check()). All seven checks use one comparison and counting method. PASS/FAIL and the summary are calculated from actual results. Common test formatting is centralized; each renderer keeps its distinct output format.

```
total++;  
boolean success = actual.equals(expected);  
if (success) {  
    passed++;  
}
```

5 - Encapsulated state (Shape and Circle fields). Private fields prevent direct external changes. The ID and radius remain fixed; only the implementation reference can be replaced through the public setter. Getters provide read access needed by T5.

```
private final String id;  
private Renderer implementation;  
  
private final int radius;
```

4. Runtime replacement on the same object

In Main.checkRuntimeSwitch(), original and circle refer to the same Circle. The program records its ID, radius and vector output, replaces the renderer, and then records the raster output. The expression circle == original proves identity; comparisons with the saved ID and radius prove that domain data did not change.

```
Circle original = circle;  
String before = circle.execute();  
circle.setImplementation(new RasterRenderer());  
String after = circle.execute();  
boolean sameObject = circle == original;
```

T5 passes only when object identity and domain-state checks are true and both output strings match the expected rendering methods. The client creates concrete renderers; the abstraction only knows Renderer.

5. Independent extension and tradeoff

The working base was committed before adding the third renderer. Base commit: 5f7616b3e8051a7bd7edf48c42db5840935d3fb5. The extension source commit is ab7d11f. It adds AsciiRenderer and T6-T7 in Main; sources.txt lists the new file. Shape, Circle, Square, Renderer, VectorRenderer and RasterRenderer remain unchanged.

extension.diff compares the base commit with the extension under src/. This is evidence that adding a rendering implementation did not require changing either shape. A tradeoff is that adding a new shape operation, such as a triangle, would require changing Renderer and all renderers. Bridge supports independence along the selected dimensions, but its operation contract still needs maintenance.

6. Bridge and Adapter have different intentions

Bridge is designed to separate abstraction from implementation so both hierarchies can vary. Here the shape hierarchy and renderer hierarchy are connected by composition. Adapter instead makes an existing incompatible interface usable through an interface expected by a client. This project has no incompatible legacy interface to translate, so Bridge matches its purpose [2].

7. Reproducible execution and evidence

The project uses only JDK classes. Main is in the default package. From the project root, compile for Java 17 and run the non-interactive demo:

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

The actual run shown in the supplied terminal screenshot produced seven PASS results and SUMMARY: 7/7 PASS. demo-output.txt preserves the actual run. T1-T4 use the same domain data for each shape: circle radius 2 and square side 3. T5 checks identity, state and output before/after replacement. T6-T7 exercise the new implementation.

```
T1: Circle + Vector -> VECTOR circle radius=2
T2: Circle + Raster -> RASTER circle radius=2
T3: Square + Vector -> VECTOR square side=3
T4: Square + Raster -> RASTER square side=3
T5: sameObject=true; stateUnchanged=true; VECTOR -> RASTER
T6: Circle + ASCII -> ASCII circle radius=2
T7: Square + ASCII -> ASCII square side=3
```

8. Repository, submission and references

Repository: <https://github.com/dauletkyzybayan-art/BridgeAssignment3>. The exact final submission commit is supplied in Moodle after committing the documentation. The ZIP is exported from that commit and contains src/, sources.txt, README.md, report.pdf, demo-output.txt and extension.diff at its root.

[1] Software Design Patterns, Assignment 3: Bridge Pattern Instructions, supplied course assignment, 2026.

[2] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994, Bridge and Adapter patterns.

AI assistance was used for code examples, explanations and drafting this report. The student must be able to explain the implementation and demonstrate changes during the required defense.